

BƯỚC 1: Import thư viện

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report
)

from xgboost import XGBClassifier
```

```
from google.colab import files
uploaded = files.upload()

import pandas as pd
import io

df = pd.read_csv('listings_ready_change_01.csv')
df.head()
```

[Choose Files](#) listings_rea...nge_01.csv
listings_ready_change_01.csv(text/csv) - 2535381 bytes, last modified: 12/2/2025 - 100% done
Saving listings_ready_change_01.csv to listings_ready_change_01.csv

	listing_id	host_id	host_name	host_since	host_response_rate	host_acceptance_rate	host_is_superhost	host_experience	prop
0	2708.0	3008.0	Chas.	16/09/2008 0:00	1.0	1.00	t	17.0	Priv
1	2732.0	3041.0	Louise- Diane	17/09/2008 0:00	1.0	0.50	f	17.0	Priv
2	6931.0	3008.0	Chas.	16/09/2008 0:00	1.0	1.00	t	17.0	Priv
3	7874.0	21700.0	Henry	15/06/2009 0:00	1.0	0.98	t	16.0	Priv
4	8021.0	22460.0	Sherree Strong	20/06/2009 0:00	1.0	1.00	f	16.0	Priv

5 rows × 27 columns

```
df = pd.read_csv('listings_ready_change_01.csv')

df['host_is_superhost'] = df['host_is_superhost'].map({'t':1, 'f':0})
df['host_acceptance_rate'] = df['host_acceptance_rate'].astype(float)
df['host_response_rate'] = df['host_response_rate'].astype(float)

features = [
    'host_acceptance_rate',
    'reviews_per_month',
    'number_of_reviews',
    'review_scores_rating',
    'host_experience',
    'host_response_rate'
]

df = df.dropna(subset=features + ['host_is_superhost'])

X = df[features]
y = df['host_is_superhost']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
def evaluate(model, X_test, y_test):
    preds = model.predict(X_test)
    probs = model.predict_proba(X_test)[:,1]
```

```

acc = accuracy_score(y_test, preds)
pre = precision_score(y_test, preds)
rec = recall_score(y_test, preds)
f1 = f1_score(y_test, preds)
auc = roc_auc_score(y_test, probs)
cm = confusion_matrix(y_test, preds)

print("\n🔴 KẾT QUẢ ĐÁNH GIÁ MÔ HÌNH")
print("Accuracy:", round(acc,4))
print("Precision:", round(pre,4))
print("Recall:", round(rec,4))
print("F1-score:", round(f1,4))
print("ROC-AUC:", round(auc,4))

print("\n🔴 Confusion Matrix:")
print(cm)

print("\n🔴 Classification Report:")
print(classification_report(y_test, preds))

return acc, pre, rec, f1, auc, cm

```

```
print("🔴 Training XGBoost...")
```

```

model_xgb = XGBClassifier(
    n_estimators=400,
    max_depth=5,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric='logloss'
)

model_xgb.fit(X_train, y_train)

```

```
🔴 Training XGBoost...
```

```

XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=0.8, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric='logloss',
               feature_types=None, feature_weights=None, gamma=None,
               grow_policy=None, importance_type=None,
               interaction_constraints=None, learning_rate=0.05, max_bin=None,
               max_cat_threshold=None, max_cat_to_onehot=None,
               max_delta_step=None, max_depth=5, max_leaves=None,
               min_child_weight=None, missing=nan, monotone_constraints=None,
               multi_strategy=None, n_estimators=400, n_jobs=None,

```

```

acc_xgb, pre_xgb, rec_xgb, f1_xgb, auc_xgb, cm_xgb = evaluate(
    model_xgb, X_test, y_test
)

```

🔴 KẾT QUẢ ĐÁNH GIÁ MÔ HÌNH

Accuracy: 0.7789
Precision: 0.7806
Recall: 0.8772
F1-score: 0.8261
ROC-AUC: 0.8376

🔴 Confusion Matrix:

```
[[463 269]
 [134 957]]
```

🔴 Classification Report:

	precision	recall	f1-score	support
0	0.78	0.63	0.70	732
1	0.78	0.88	0.83	1091
accuracy			0.78	1823
macro avg	0.78	0.75	0.76	1823
weighted avg	0.78	0.78	0.77	1823

```
import pandas as pd
import matplotlib.pyplot as plt

# Lấy importance từ mô hình XGBoost
importance = pd.Series(model_xgb.feature_importances_, index=features)

# Sắp xếp từ quan trọng nhất → ít nhất
importance_sorted = importance.sort_values(ascending=False)

print("🔥 FEATURE IMPORTANCE - XGBOOST (TỪ MẠNH → YẾU):")
print(importance_sorted)
```

```
🔥 FEATURE IMPORTANCE - XGBOOST (TỪ MẠNH → YẾU):
review_scores_rating    0.233166
host_response_rate      0.218123
host_acceptance_rate    0.181488
reviews_per_month       0.150684
host_experience          0.114404
number_of_reviews       0.102135
dtype: float32
```

```
plt.figure(figsize=(7,5))
importance_sorted.plot(kind='bar')
plt.title("Feature Importance - XGBoost")
plt.ylabel("Mức độ ảnh hưởng")
plt.show()
```



